IV კვირის შემაჯამებელი დავალება

პროექტის თვითშეფასება და ტექნიკური აუდიტი

WMS — საწყობის მართვის სისტემა

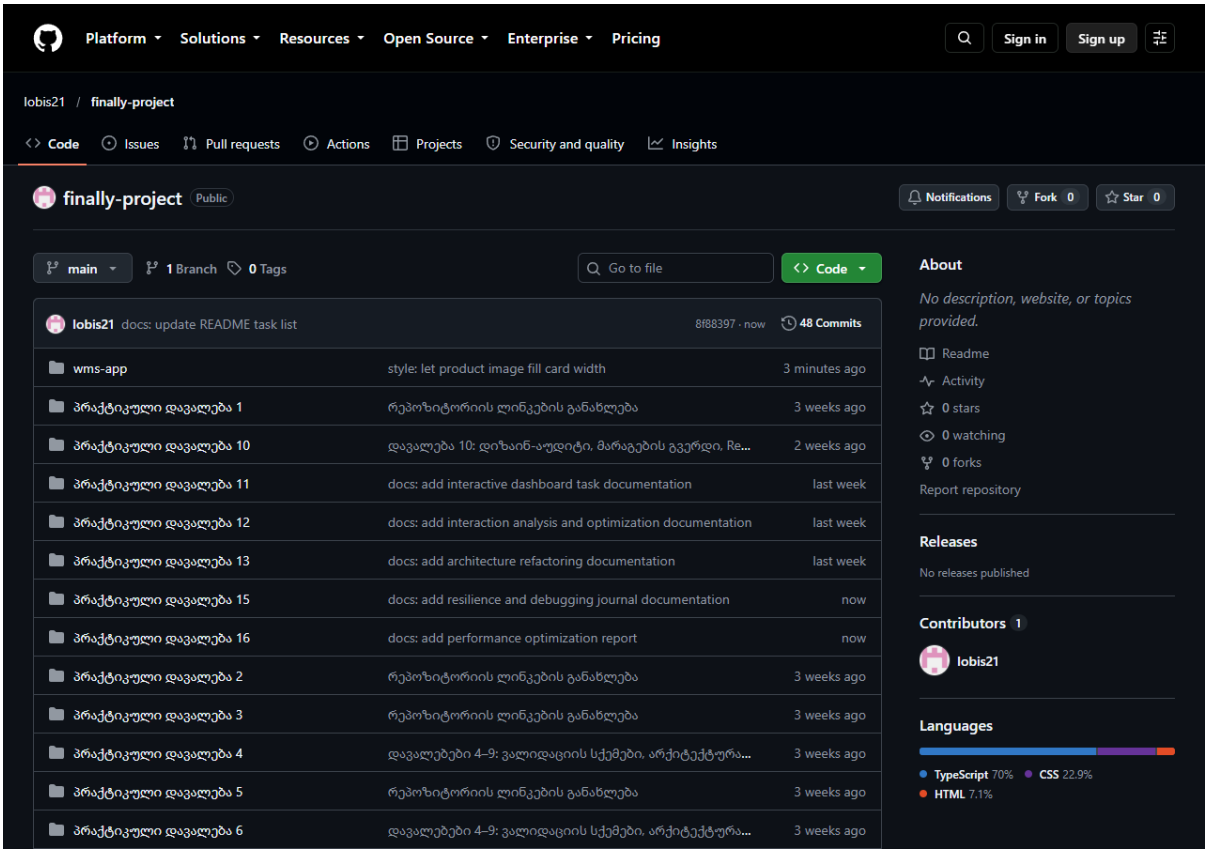
1. სტუდენტი	იობი შარაშენიძე
2. GitHub რეპოზიტორია	https://github.com/Iobis21/finally-project
3. Live Demo	არ არის (ლოკალური გაშვება: npm install → npm run dev)
კანბან დაფა	https://github.com/users/Iobis21/projects/1
სატესტო ანგარიში	`manager@wms.ge / Wms12345!`

1. დაგეგმვა და Git ვერსიების მართვა

> კითხვა: როგორ არის დოკუმენტირებული პროექტის სტრუქტურა (README.md) და რა პრინციპით მართავდით ბრენჩებსა და კომიტებს?

თემა: როლებზე დაფუძნებული საწყობის მართვის სისტემა (WMS) ოთხი როლით — საწყობის მენეჯერი, შესყიდვები, ოპერატორი, ლოჯისტიკა. **სტეკი:** React 19, TypeScript, Vite, Tailwind CSS v4, React Router, React Hook Form + Zod, Lucide Icons, json-server + json-server-auth (JWT).

README.md-ში აღწერილია გაშვება, სატესტო ანგარიშები, ტექნოლოგიები და ყველა დავალების ბმული; თითო დავალებას საკუთარი დოკუმენტი აქვს (პრაქტიკული დავალება N/). **Git:** ვმუშაობ `main` ბრენჩზე, მაგრამ სამუშაოს ვყოფ პატარა, **ატომურ კომიტებად** Conventional Commits სტანდარტით (`feat:`, `fix:`, `refactor:`, `perf:`, `style:`, `docs:`) — 48+ კომიტი; თასქები იმართება GitHub Projects-ის კანბან დაფაზე.



GitHub რეპოზიტორიის მთავარი გვერდი (1 ბრენჩი, 48 კომიტი)

```
git log --oneline --graph
```

```
* 8f88397 docs: update README task list
* 41a17e0 docs: add performance optimization report
* c18c475 docs: add resilience and debugging journal documentation
* f084b8f style: let product image fill card width
* 58cac23 fix: ignore stale item responses and use unique console timers
* 1d31c94 fix: make ErrorBoundary crash test survive React render retry
* 295f029 perf: add category images as WebP with lazy loading
* 06408f3 perf: lazy load route pages with React.lazy and Suspense
* 03f65ca perf: memoize inventory rows, cards and callbacks with React.memo and useCallback
* e82409a feat: add stock write-off with negative stock and race condition protection
* 9ece1ad feat: fall back to cached or demo data when server is offline
* ab185a7 feat: wrap pages in ErrorBoundary with retry fallback
* 5d35d79 docs: document authentication, roles and test accounts
* dd18d60 style: apply prettier formatting
* 346512f feat: allow item add/edit/delete only for manager role
* ffd480f feat: protect app routes and require login
* 1635a6e feat: add users page for role assignment and account blocking
* 4eb1a9e feat: add login and register pages with validation
* cd9fc81 refactor: move FormField into shared components
* a39877b feat: replace role switcher with AuthContext and route guards
* f7f5c9c feat: add JWT auth server, role config and auth/user services
* 55d4407 docs: update README task list
* ef33cb7 docs: add architecture refactoring documentation
* 1367010 docs: add interaction analysis and optimization documentation
* a30c18b docs: add interactive dashboard task documentation
```

git log --oneline --graph — ატომური კომიტები Conventional Commits-ით

2. სემანტიკური HTML5 და მოდულური არქიტექტურა

> **კითხვა:** რომელი სემანტიკური HTML5 თეგები გამოიყენეთ და როგორ დაიცავით კომპონენტების მოდულურობა (SOLID / DRY)?

ლეიაუთი აწყობილია სემანტიკური თეგებით: `<header>` (Header), `<aside>` + `<nav>` (Sidebar), `<main>` (გვერდის შიგთავსი), `<section>` (სტატისტიკა, პროდუქტები), `<article>` (თითო კარტა), `<footer>`; ცხრილში `<th scope>`, ბარათებში `<dl>/<dt>/<dd>`. ცალკე დავალებაში აწყობილია სუფთა HTML5 სკელეტი `figure`, `time`, `address` თეგებით.

საქაღალდეები: `components/` — UI, `hooks/` — ლოგიკა (useItems, useInventoryFilters, useDebounce), `services/` — API (fetch მხოლოდ აქ), `types/` — ინტერფეისები. **SRP:** მარაგების გვერდი დაიყო hook-ებად და 5 კომპონენტად; **OCP:** Modal და Button იღებენ children/variant-ს; **DRY:** ერთი request() ფუნქცია, ერთი StatusBadge და FormField.

wms-app/src — ფაილების სტრუქტურა

```
src/
  components
    ConfirmDialog
    EmptyState
    ErrorBoundary
    Footer
    FormField
    Header
    ItemForm
    Layout
    Loader
    OfflineBanner
    ProductCard
    Sidebar
    StatCard
    StatusBadge
    WriteOffForm
    table
    ui
      Button
      Modal
  config
  context
  data
  hooks
  pages
    auth
    dashboard
    errors
    inventory
      components
    users
  routes
  schemas
  services
  types
  utils
```

src/ საქაღალდეების სტრუქტურა

src/components/Layout/Layout.tsx

```tsx
import { Suspense, useEffect, useState } from 'react'
import { Outlet, useLocation, useNavigate } from 'react-router'
import ErrorBoundary from '../ErrorBoundary/ErrorBoundary'
import Header from '../Header/Header'
import Sidebar from '../Sidebar/Sidebar'
import Footer from '../Footer/Footer'
import { SkeletonGrid } from '../Loader/Loader'
import './Layout.css'

function Layout() {
  // მობილურზე Sidebar-ის გახსნა/დახურვა
  const [isMenuOpen, setIsMenuOpen] = useState(false)
  const location = useLocation()
  const navigate = useNavigate()

  // გვერდის შეცვლისას მენიუ იხურება
  useEffect(() => {
    setIsMenuOpen(false)
  }, [location.pathname])

  // ღია მენიუ: Esc ხურავს, ფუკნა გვერდი არ სქროლდება
  useEffect(() => {
    if (!isMenuOpen) return

    function handleKey(e: KeyboardEvent) {
      if (e.key === 'Escape') setIsMenuOpen(false)
    }

    document.addEventListener('keydown', handleKey)
    document.body.style.overflow = 'hidden'

    return () => {
      document.removeEventListener('keydown', handleKey)
      document.body.style.overflow = ''
    }
  }, [isMenuOpen])

  return (
    <div className="layout">
      <Header isMenuOpen={isMenuOpen} onMenuClick={() => setIsMenuOpen(!isMenuOpen)} />
      <Sidebar isOpen={isMenuOpen} onClose={() => setIsMenuOpen(false)} />
      <main className="layout__main">
        {/* თუ გვერდი „გატყდება", Header და მენიუ მაინც მუშაობს */}
        <ErrorBoundary
          resetKey={location.pathname}
          onReset={() => navigate(location.pathname, { replace: true })}
        >
          <Suspense fallback={<SkeletonGrid count={4} />}>
            <Outlet />
          </Suspense>
        </ErrorBoundary>
      </main>
      <Footer />
    </div>
  )
}

export default Layout
```

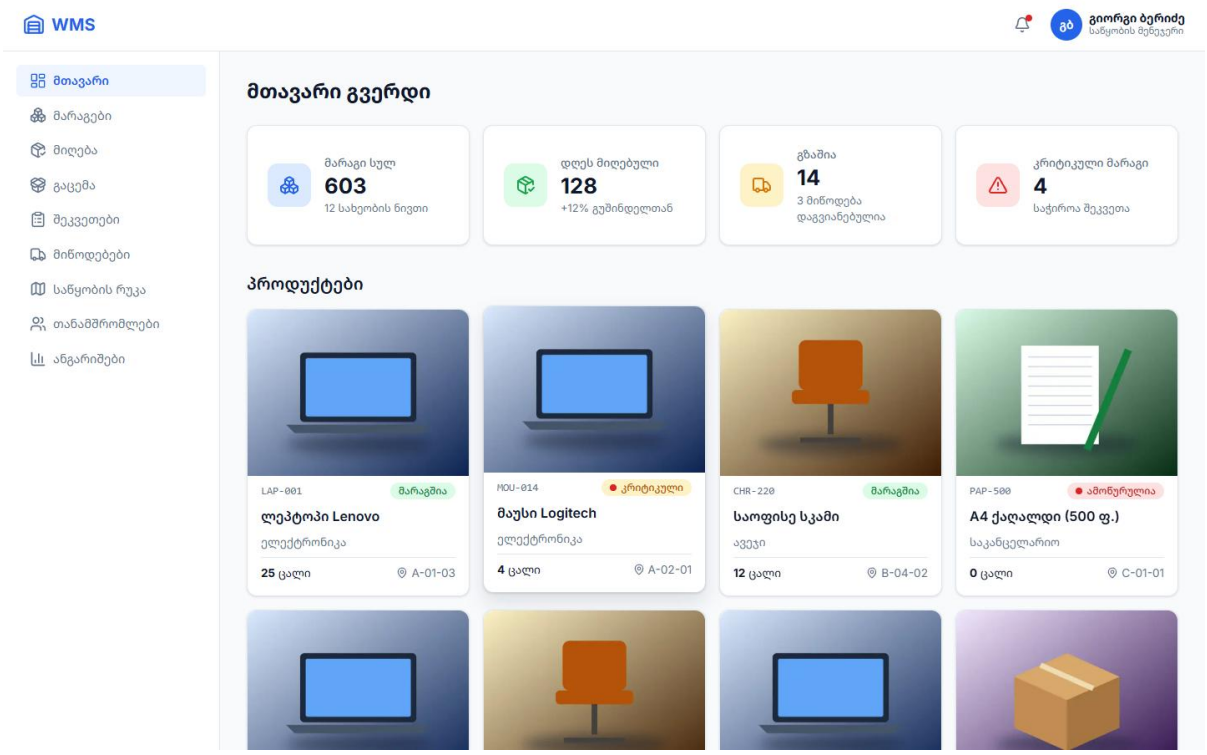
Layout.tsx — მთავარი ლეიაუთი (Header, Sidebar, main + ErrorBoundary, Footer)

3. დიზაინ-სისტემა და რესპონსიულობა

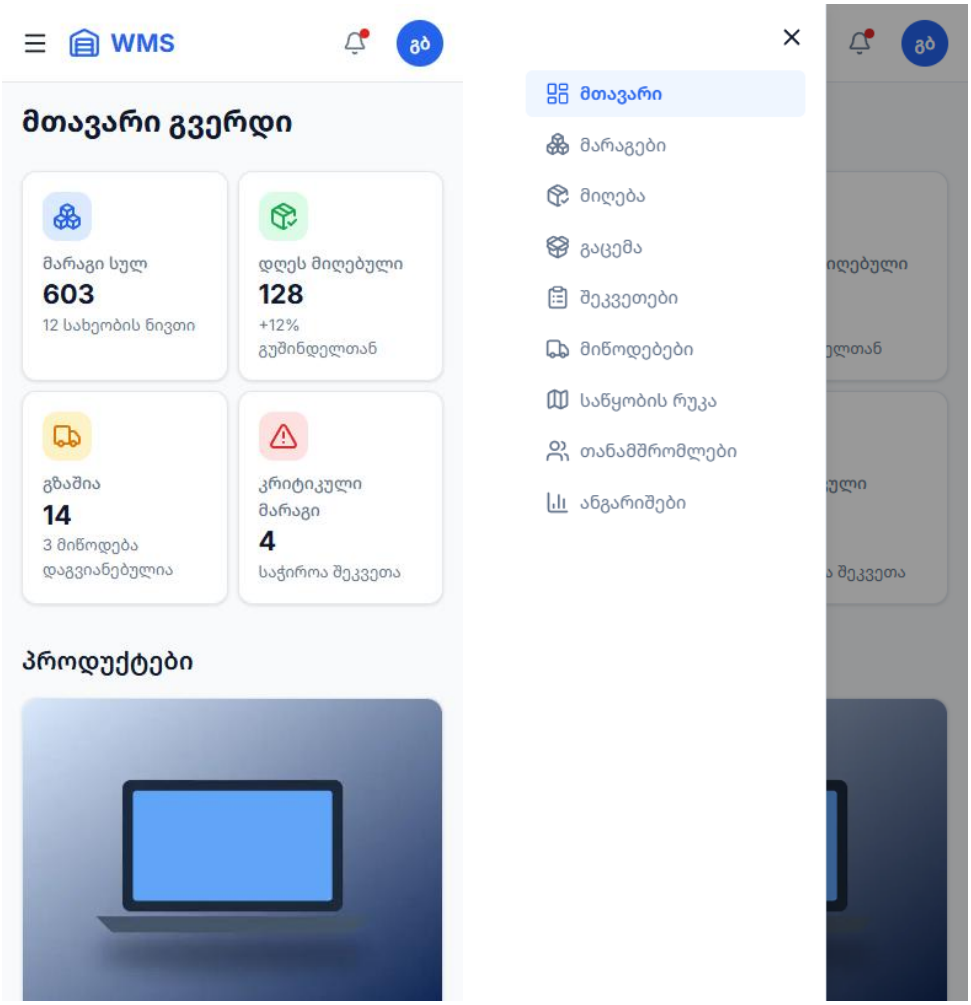
> **კითხვა:** როგორ მუშაობს ინტერფეისი 375px / 768px / 1440px ეკრანზე და როგორ გადაწყვიტეთ მობილური ნავიგაცია და ბადეები?

ყველა ფერი, დაშორება (8px ბადე: 4/8/16/24/32), რადიუსი და ჩრდილი აღწერილია CSS ცვლადებად `:root`-ში და Tailwind-ის `@theme`-ში. გვერდის ჩონჩხი — **CSS Grid** (`260px 1fr` + grid-template-areas), კომპონენტების შიგნით — **Flexbox**; კარტები `repeat(auto-fit, minmax(220px, 1fr))`-ით თავისით გადადიან ახალ ხაზზე.

768px-ზე ქვემოთ Grid ხდება 1-სვეტიანი, Sidebar იმალება და იხსნება **ჰამბურგერ ☰** ღილაკით (transform: translateX), ფონი მუქდება, Esc ხურავს; 900px-ზე ქვემოთ მარაგების ცხრილი იქცევა ბარათებად. 375px-ზე ჰორიზონტალური სქროლი არ არის.



დესკტოპი — 1440px



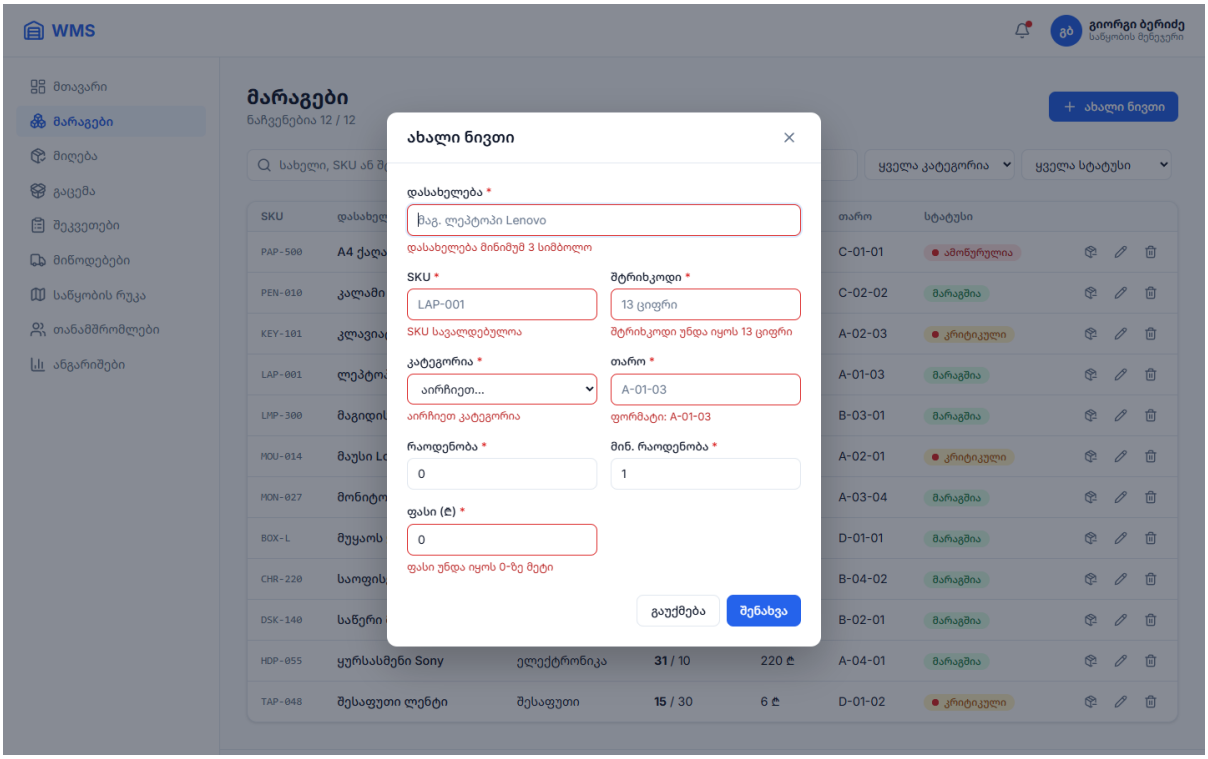
მობილური — 375px: 1-სვეტიანი ბადე და გახსნილი ჰამბურგერ მენიუ

4. ინტერაქტიულობა, მოდალები და წარმადობა

> **კითხვა:** როგორ არის ოპტიმიზებული ძიება (Debounce), როგორ მუშაობს მოდალი (Portal) და რა მიკრო-ინტერაქციები დანერგეთ?

Debounce: ძებნა იწყება მხოლოდ მაშინ, როცა მომხმარებელი 300ms-ით შეწყვეტს ბეჭდვას — „ლეპტოპის" აკრეფისას 7-ის ნაცვლად ერთი ფილტრაცია ხდება. **Portal:** `createPortal(..., document.body)` მოდალს ხატავს <body>-ში, ამიტომ მას მშობლის overflow / z-index არ ზღუდავს; იხურება ✕-ით, ფონზე კლიკით და Esc-ით, ფორმას აქვს Zod ვალიდაცია.

მიკრო-ინტერაქციები მხოლოდ `transform` და `opacity`-ით (GPU): ბარათი hover-ზე აიწევა (translateY), ჩრდილი ::after-ის opacity-ით ჩნდება, კრიტიკულ მარაგს აქვს წითელი პულსირებადი წერტილი (scale + opacity), მოდალი შემოდის ქვემოდან. **წარმადობა:** React.memo, useCallback, useMemo, React.lazy (მთავარი bundle 429 → 267 kB), WebP სურათები loading="lazy"-ით (8.3 MB → 8.9 KB).



მოდალური ფანჯარა (Portal) — ფორმის ვალიდაცია

src/hooks/useDebounce.ts

```
import { useEffect, useState } from 'react'

// Debounce: მნიშვნელობა განახლდება მხოლოდ მაშინ, როცა მომხმარებელი `delay` მილიწამით შეწყვეტს ბეჭდვას.
// ყოველ ახალ ასოზე წინა ტაიმერი უქმდება და ახალი იწყება.
export function useDebounce<T>(value: T, delay = 300): T {
  const [debouncedValue, setDebouncedValue] = useState(value)

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay)
    return () => clearTimeout(timer)
  }, [value, delay])

  return debouncedValue
}
```

useDebounce.ts

src/components/ui/Modal/Modal.tsx — createPortal

```
  if (!isOpen) return null

  // Portal: მოდალი იხატება <body>-ში და არა იქ, სადაც კომპონენტი დგას
  return createPortal(
    <div className="modal__overlay" onClick={onClose}>
      <div
        ref={dialogRef}
        className="modal"
        role="dialog"
        aria-modal="true"
        aria-labelledby="modal-title"
        onClick={(e) => e.stopPropagation()} // შიგნით დაჭერა მოდალს არ ხურავს
      >
        <header className="modal__header">
          <h2 id="modal-title" className="modal__title">
            {title}
          </h2>
          <button className="modal__close" onClick={onClose} aria-label="დახურვა">
            <X size={20} />
          </button>
        </header>
        <div className="modal__body">{children}</div>
      </div>
    </div>,
    document.body,
  )
}

export default Modal
```

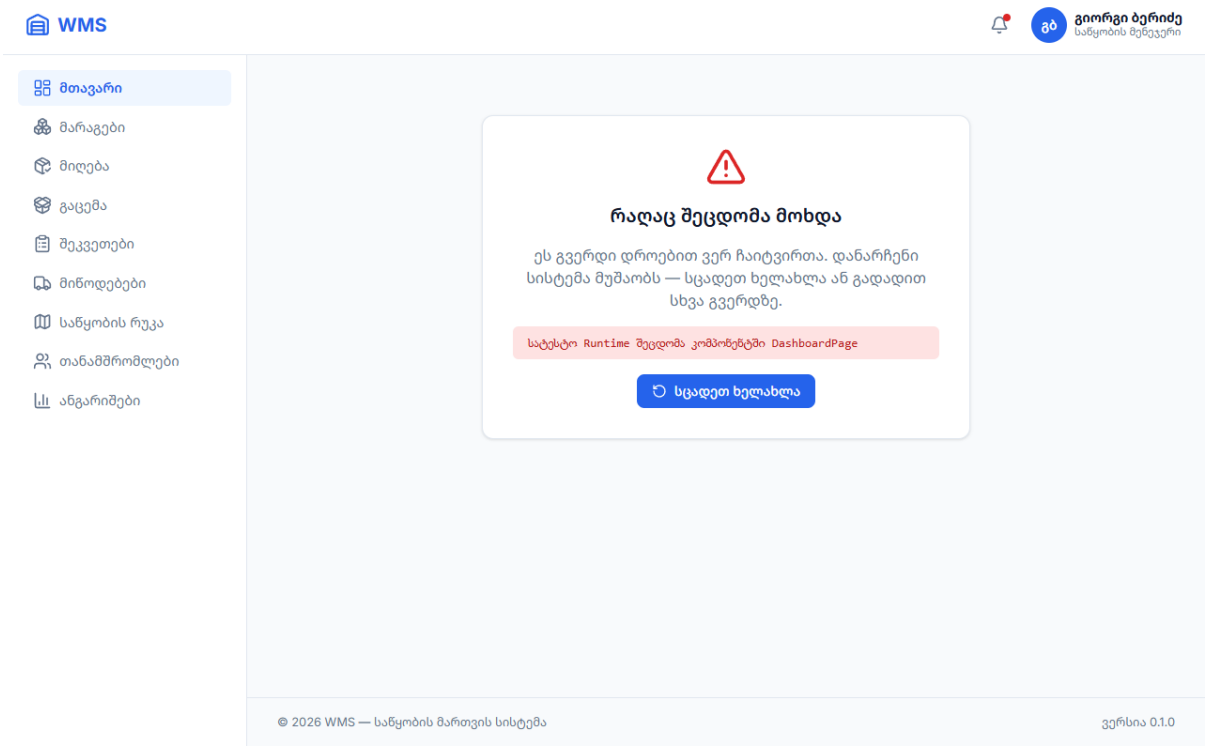
Modal.tsx — ReactDOM.createPortal

5. მდგრადობა (Resilience) და Debugging

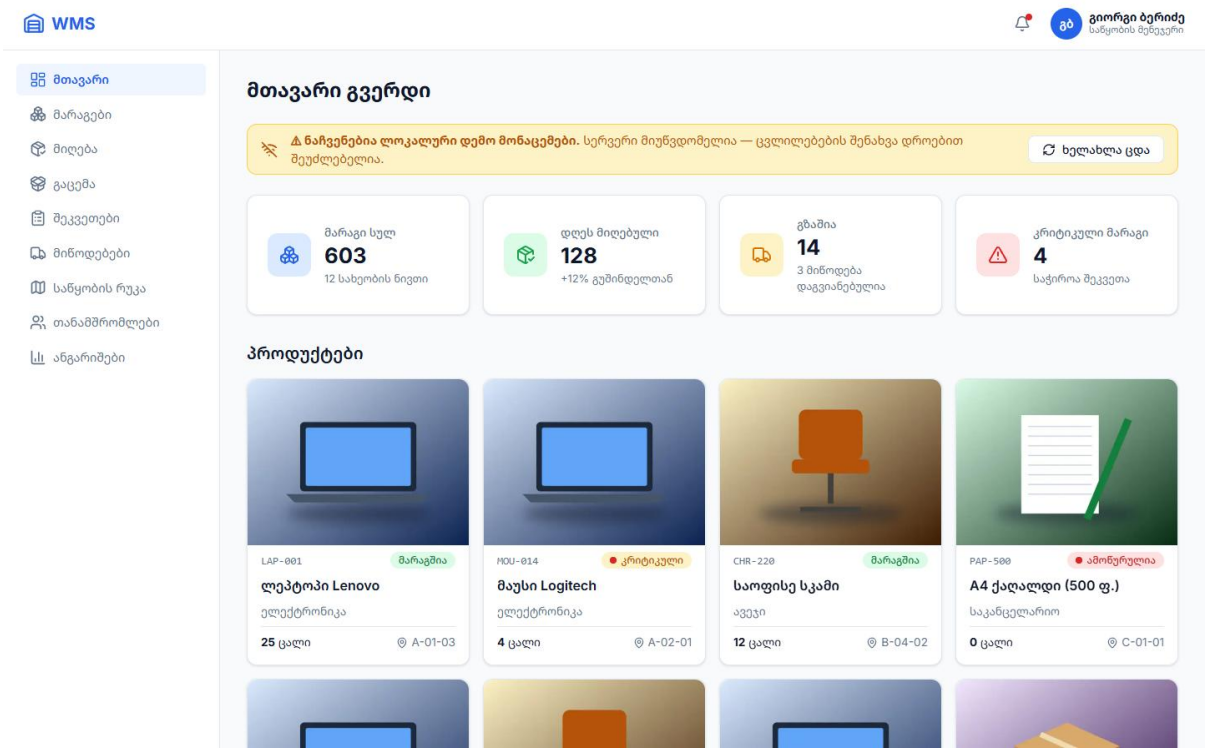
> **კითხვა:** როგორ არის დაცული აპლიკაცია სერვერის გათიშვის, ორმაგი დაჭერის ან Runtime შეცდომის დროს?

ErrorBoundary: ყველა გვერდი შეფუთულია ErrorBoundary-ში — Runtime შეცდომისას თეთრი ეკრანის ნაცვლად ჩანს Fallback და ღილაკი „სცადეთ ხელახლა"; Header და მენიუ აგრძელებს მუშაობას. **ოფლაინ რეჟიმი:** თუ სერვერი 500-ს აბრუნებს ან ინტერნეტი გაწყდა, აპი იყენებს ბოლო ქეშს (localStorage) ან დემო მონაცემებს და აჩვენებს ბანერს „⚠ ნაჩვენებია ლოკალური მონაცემები"; ცვლილებების ღილაკები ამ დროს იმალება.

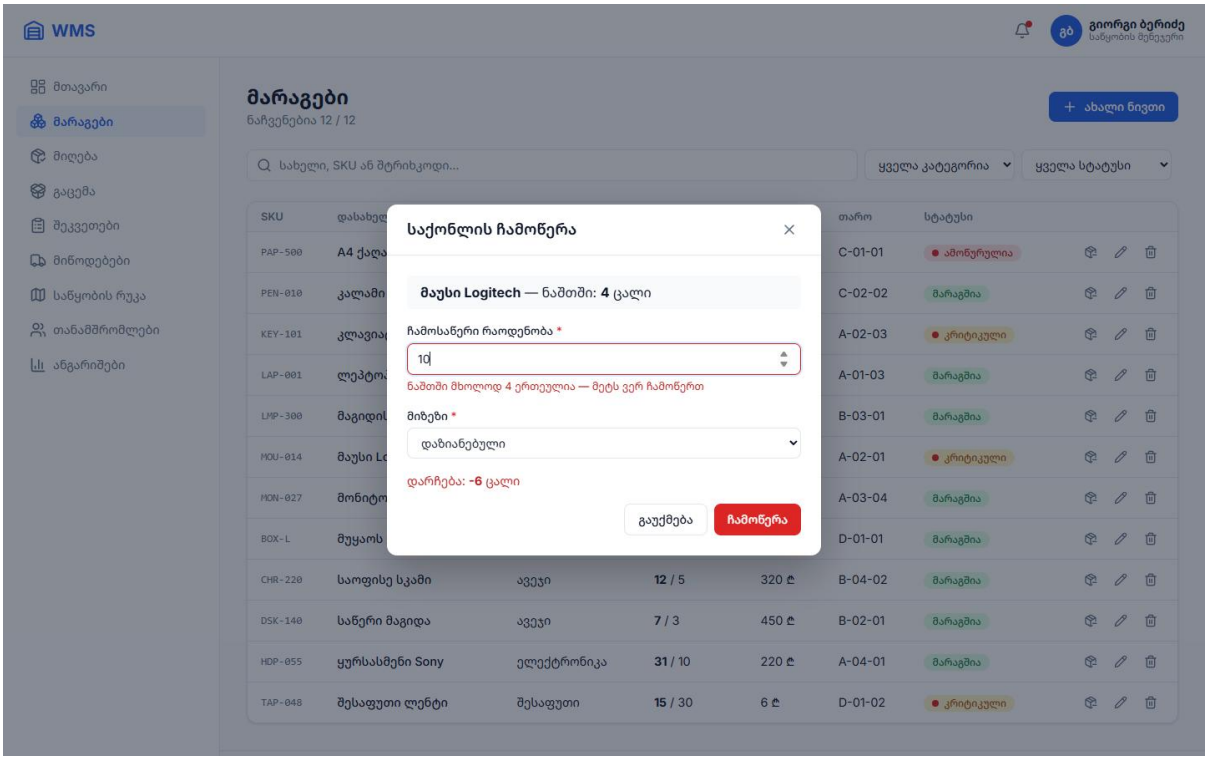
Double-Click: გაგზავნისას ღილაკი გადადის disabled + „ინახება..." რეჟიმში. **Fail-Fast:** Zod ფორმას სერვერამდე ამოწმებს — მაგ. ჩამოწერა ნაშთზე მეტი ვერ იქნება (უარყოფითი ნაშთის დაცვა), ჩაწერამდე კი სერვერიდან მოწმდება უახლესი ნაშთი (Race Condition). კოდში გამოიყენება `console.table()` და `console.time()`; დებაგინგის ჟურნალში აღწერილია 3 რეალური ბაგი 6-საფეხურიანი სტრუქტურით.



ErrorBoundary — Fallback UI და „სცადეთ ხელახლა"



სერვერის გათიშვა — ლოკალური მონაცემების ბანერი



უარყოფითი ნაშთის დაცვა — Zod Fail-Fast ვალიდაცია